

Prompt Ladder – Improving a FastAPI Task API Prompt

Name: Thillai Eswari

Track: FlyRank AI – Backend AI Engineer

Assignment: AI Fluency – Week 2

Objective

The goal of this exercise is to improve a weak prompt one step at a time and observe how each individual change affects the AI's response. Only one prompt-engineering layer is added in each version so it is clear which change improves the output.

Baseline

Prompt

Write backend code for a task API.

Representative Output (Excerpt)

```
const express = require("express");

const app = express();

app.use(express.json());

let tasks = [];

app.get("/tasks", (req, res) => {

  res.json(tasks);

});
```

Notes

What changed in the prompt

This is the starting point before any improvements were made.

What improved in the output

The AI generated a basic backend API with REST endpoints, showing that it understood the general request.

What still failed

The prompt was too vague. The AI assumed Express.js instead of FastAPI and did not know the project requirements or expected features.

What I would try next

Specify the framework and make the goal clearer.

Version 1 – Clearer Goal

Prompt

Write backend code for a task management API using FastAPI.

Representative Output (Excerpt)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/tasks")
def get_tasks():
    return []
```

Notes

What changed in the prompt

I clearly specified that the project should use FastAPI.

What improved in the output

The response focused on FastAPI instead of a generic backend implementation, making it more relevant to my backend internship work.

What still failed

Simply specifying the framework was not enough. The AI still did not know the assignment requirements or the level of explanation I needed.

What I would try next

Add the intended audience.

Version 2 – Defined Audience

Prompt

Write backend code for a task management API using FastAPI for a beginner backend developer learning REST APIs.

Representative Output (Excerpt)

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# Create a simple Task model

class Task(BaseModel):

    title: str
```

Notes

What changed in the prompt

I added the intended audience.

What improved in the output

The explanation became easier to understand. The AI included simpler examples and beginner-friendly guidance instead of assuming advanced knowledge.

What still failed

The response still did not know the specific internship requirements.

What I would try next

Provide project context.

Version 3 – Real Context

Prompt

Write backend code for a task management API using FastAPI for a beginner backend developer learning REST APIs.

This is an internship assignment that requires CRUD operations and Swagger API documentation.

Representative Output (Excerpt)

```
@app.post("/tasks")

def create_task(task: Task):

    ...
```

```
@app.put("/tasks/{task_id}")  
  
def update_task(task_id: int):  
  
    ...
```

Notes

What changed in the prompt

I added the real project context and assignment requirements.

What improved in the output

The AI generated CRUD endpoints and included Swagger support because it understood the actual purpose of the project. The response became much more relevant.

What still failed

Although the content improved, the answer was still presented as one long response.

What I would try next

Specify an output format.

Version 4 – Specified Output Format

Prompt

Write backend code for a task management API using FastAPI for a beginner backend developer learning REST APIs.

This is an internship assignment that requires CRUD operations and Swagger API documentation.

Provide the response in this format:

1. Project structure
2. Required files
3. Code
4. API endpoints
5. Testing steps

Representative Output (Excerpt)

1. Project Structure
2. Files
3. API Endpoints
4. Testing
5. Next Steps

Notes

What changed in the prompt

I specified how I wanted the response to be organized.

What improved in the output

Instead of returning one large response, the AI organized the information into clear sections, making it much easier to follow and implement.

What still failed

The AI generated the solution but did not verify whether every CRUD operation had been covered.

What I would try next

Ask the AI to verify its own work.

Version 5 – Verification Requirement

Prompt: Write backend code for a task management API using FastAPI for a beginner backend developer learning REST APIs.

This is an internship assignment that requires CRUD operations and Swagger API documentation.

Provide the response in this format:

1. Project structure
2. Required files
3. Code
4. API endpoints
5. Testing steps

Before finishing, verify that every CRUD operation is implemented correctly and mention possible errors.

Representative Output (Excerpt)

- ✓ CRUD endpoints verified
- ✓ Validation included
- ✓ Testing steps provided
- ✓ Common errors explained

Notes

What changed in the prompt

I added a verification requirement.

What improved in the output

The AI reviewed its own solution, included testing suggestions, and highlighted common mistakes. The final response was more complete and practical than the previous versions.

What still failed

The generated code should still be tested in a real development environment before deployment.

What I would try next

Test the generated code and refine the prompt further based on real implementation results.

Final Reusable Prompt

The following prompt combines all the improvements from the previous versions into a reusable prompt that another backend engineering intern could use.

Act as a backend development mentor.

Help me build a task management API using FastAPI.

This is a backend internship assignment for a beginner learning REST APIs.

The project must include CRUD operations and Swagger documentation.

Provide the response in the following format:

1. Project structure
2. Required files
3. Complete implementation
4. API endpoints
5. Testing instructions
6. Common errors and solutions

Before finishing, verify that every CRUD operation has been implemented correctly and explain any assumptions you made.

Reflection

This exercise helped me understand how small prompt changes can significantly improve AI responses. At first, I thought simply mentioning FastAPI would be enough, but I realized that adding context, defining the audience, and specifying the output format made a much bigger difference. Comparing each version taught me which prompt changes actually improved the quality of the response instead of just making the prompt longer.